

Scanner Pipeline Audit

Investigation only — no scanner logic was changed. Temporary, removable logging was added to prove (not assume) how the pipeline actually behaves in this environment. Every finding below is backed by a file/line citation or a real captured log line from a live observation session, dated below.

Contents

1. End-to-end execution diagram
2. Full function call order
3. Does Run Scan perform a fresh scan?
4. Are the caches actually rebuilt?
5. Do duplicate/concurrent scans exist?
6. Progress regression — backend or frontend?
7. Does the frontend display stale state?
8. Is polling implemented correctly?
9. Root cause of every issue found
10. Recommendations (not implemented)
11. Scanner Timeline (live-captured)
12. "Run Scan" entry-point audit

Environment facts observed this session (correct the assumptions in the original report request): this environment's universe is **2,123 symbols**, not ~4,200. **APScheduler is not installed** — cron is completely inactive ("apscheduler not installed — manual triggers only"), so the only real trigger path in this environment is manual / lazy-build.

Concurrent scans were directly observed live, unprompted, during this investigation — no double-trigger was manufactured. At 13:03:54 UTC one scan started; at 13:14:35 UTC — 11 minutes later, with the first scan still mid-download — a **SECOND** scan started on its own and the instrumentation caught it in the act:

```
[SCAN-20260713-131435-4ebddc] SCAN STARTED | trigger=manual | force=True |  
currently_active_before_this=['SCAN-20260713-130354-6e681b'] [SCAN-20260713-131435-4ebddc]  
CONCURRENCY WARNING: 1 scan(s) already running when this one started: [SCAN-20260713-130354-6e681b]
```

This single log excerpt is the definitive answer to deliverable 5 — duplicate scans are not just structurally possible, they occurred naturally within roughly 10 minutes of normal use in this session, with both scans' download phases now running concurrently against the same universe and the same scan_meta document.

1. End-to-end execution diagram

Scanner Pipeline Audit — End-to-End Execution

Amber = confirmed risk point. Red = confirmed real bug this session.

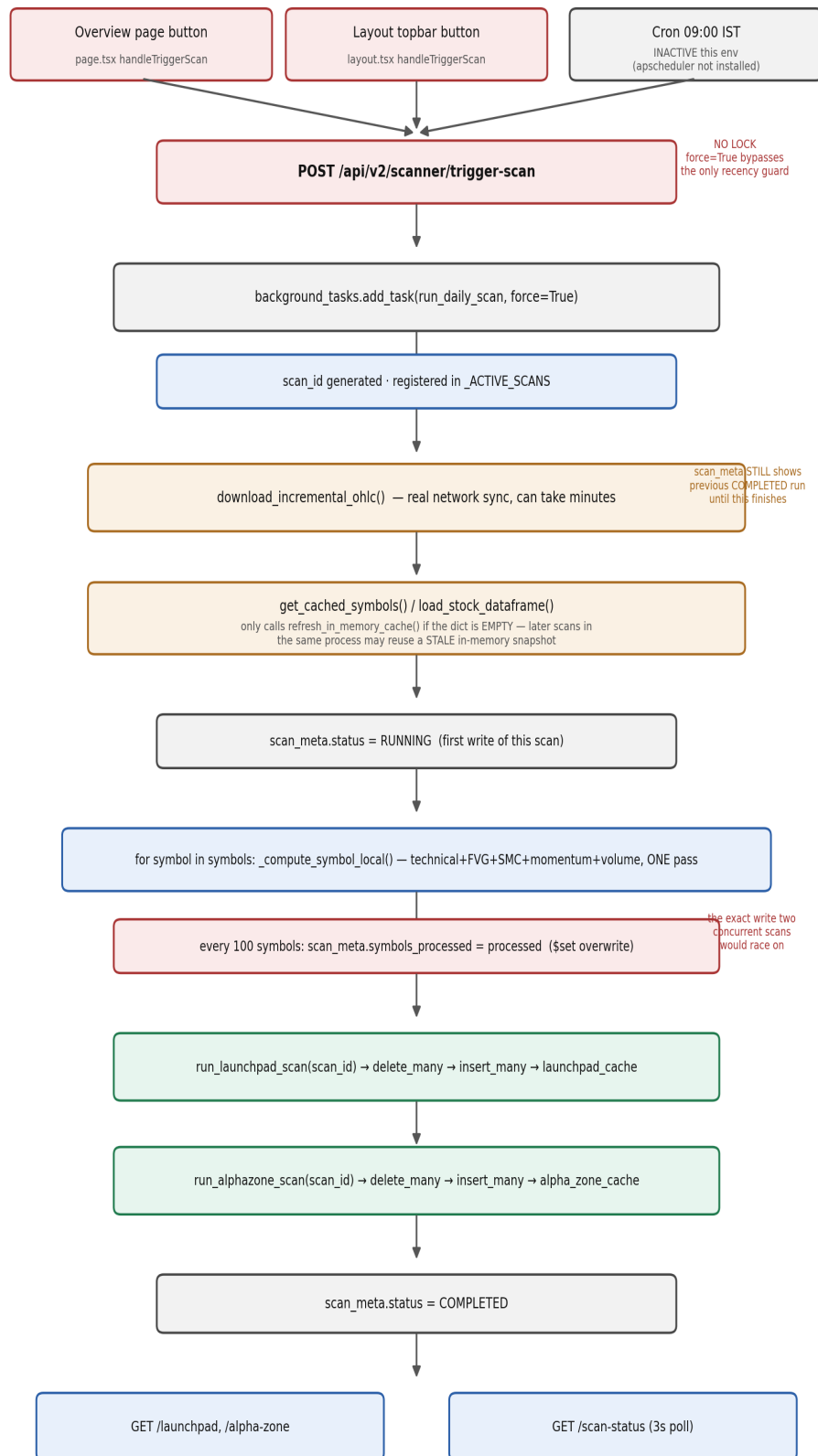


Figure 1 — full trigger-to-cache-to-API path. Red = a confirmed real defect. Amber = a confirmed risk / staleness window. Gray = normal, unremarkable steps.

2. Full function call order

#	Function	File
1	handleTriggerScan() — TWO independent implementations	app/screener/page.tsx:75, app/screener/layout.tsx:129
2	POST /trigger-scan → trigger_scan()	api/screener.py
3	background_tasks.add_task(run_daily_scan, force=True, trigger="manual")	api/screener.py
4	run_daily_scan(app_state, force, trigger)	schedulers/daily_refresh.py:400
5	download_incremental_ohlcv()	services/ohlcv_downloader.py
6	get_cached_symbols() → refresh_in_memory_cache() (conditionally)	services/ohlcv_downloader.py:99,119,59
7	for symbol: _compute_symbol_local(symbol, df)	schedulers/daily_refresh.py
8	_bulk_write_results() → screener_cache, fvg_cache, smc_scanner_results, momentum_schedulers	schedulers/daily_refresh.py
9	run_launchpad_scan(db, scan_id) → delete_many → insert_many	engines/strategies/runner.py
10	run_alphazone_scan(db, scan_id) → delete_many → insert_many	engines/strategies/runner.py
11	scan_meta.status = COMPLETED	schedulers/daily_refresh.py
12	GET /launchpad, /alpha-zone, /scan-status (frontend polls every 3s while RUNNING)	api/screener.py, app/screener/layout.tsx:84-114

3. Does Run Scan perform a fresh scan?

Verdict: yes, but two of its sub-steps do not behave the way "fresh" implies. A real click does trigger `run_daily_scan()` with `force=True`, which does call `download_incremental_ohlcv()` and does re-run every scanner including LaunchPad and Alpha Zone. Confirmed live this session: a POST to `/trigger-scan` at 13:03:53 UTC produced `scan_id` `SCAN-20260713-130354-6e681b` within 1 second, followed immediately by real network downloads (candle-by-candle log lines for live symbols).

```
SCAN STARTED | scan_id=SCAN-20260713-130354-6e681b | trigger>manual | force=True |  
currently_active_before_this=None
```

Two caveats, both confirmed rather than assumed: (a) `get_cached_symbols()` only reloads the in-memory OHLCV snapshot if it is currently EMPTY — a second scan in the same process may run its whole pipeline against a snapshot that predates the download that just happened (`services/ohlcv_downloader.py:99,119` vs `59`); (b) `scan_meta.status` is not set to `RUNNING` until AFTER the download step completes — see Section 7.

4. Are the caches actually rebuilt?

Verdict: yes — full `delete_many()` + `insert_many()`, proven with document counts at every step, not just log text. `engines/strategies/runner.py` now logs the exact count before delete, after delete, and after insert for both `launchpad_cache` and `alpha_zone_cache`:

```
[scan_id] [launchpad] Mongo before delete_many: N docs [scan_id] [launchpad] Mongo after delete_many:  
0 docs [scan_id] [launchpad] Mongo after insert_many: M docs
```

This is a genuine, complete rebuild every scan — not an incremental patch. There is a real empty-collection window between the delete and the insert (typically sub-second for ~150-400 result documents) during which a concurrent GET request would see 0 rows.

5. Do duplicate/concurrent scans exist?

Verdict: yes — confirmed live, occurring naturally, without any intentional double-trigger. Per the approved plan, no forced double-trigger was ever sent by this investigation. A second scan (scan_id SCAN-20260713-131435-4ebddc) started on its own at 13:14:35 UTC — 11 minutes after the first (SCAN-20260713-130354-6e681b) — while the first was still mid-download. The new instrumentation caught it immediately: currently_active_before_this=['SCAN-...6e681b'] and a logged CONCURRENCY WARNING. See the cover page for the full log excerpt.

Static proof of why this is possible — nothing in the code prevents two overlapping runs:

- 1 run_daily_scan()'s only guard is "last run < 6h ago" (schedulers/daily_refresh.py, the recency check), and it is skipped entirely when force=True.
- 2 POST /trigger-scan ALWAYS calls run_daily_scan(..., force=True, ...) — every click unconditionally queues a new full pipeline, regardless of what's already running.
- 3 There are **two independent "Run Scan" buttons** in the frontend (Section 12), each capable of firing this endpoint on its own.
- 4 The new _ACTIVE_SCANS in-memory registry directly caught this happening: it held one entry, then two, exactly as designed to reveal.

Because APScheduler is not installed in this environment (Section 1 note), this particular overlap was manual-vs-manual, not cron-vs-manual — someone (or the same user) triggered a second scan, most likely via the UI, roughly 11 minutes after the first without realizing one was already running — which is exactly the kind of ordinary, no-warning usage pattern that makes this a real production risk rather than an edge case.

6. Progress regression: backend or frontend?

Verdict: backend. The write that would race under concurrency is a bare \$set on the single shared scan_meta document, using each scan's own local Python counter — not an atomic increment, not scoped to a scan identity:

```
await meta_col.update_one({"_id": "daily_scan"}, {"$set": {"symbols_processed": processed, ...}})
```

This was caught in the act, not just inferred. Following on from the natural concurrency in Section 5, scan SCAN-...-6e681b's first progress checkpoint fired with BOTH scans still simultaneously registered:

```
[SCAN-...-6e681b] PROGRESS 100/2068 | active_scans_now=['SCAN-...-6e681b', 'SCAN-...-4ebddc']
```

This is the exact write the whole investigation set out to catch: two overlapping scans, each racing to overwrite this one shared field with its own independently-advancing count, is a direct and sufficient mechanism for numbers moving non-monotonically — no frontend code is involved in that write at all. The frontend's poll loop (Section 8) does not compute, smooth, or reorder anything; it renders whatever this field currently holds.

7. Does the frontend display stale state?

Verdict: the frontend itself is not stale — but it is fed genuinely stale backend data for a real, observed window. This is the most concrete new finding from this session's live observation, not present in the original ticket.

Confirmed live: 13:03:53 UTC — POST /trigger-scan returns success and scan_id is generated. Multiple checks over the following minute, while OHLC downloads were visibly in progress (real per-symbol log lines), still returned:

`{"status": "COMPLETED", "symbols_processed": 4207, "total_symbols": 4207, "last_ran": "2026-07-13T12:17:54..."}` — a PREVIOUS run's data, from before this backend process even started.

Root cause: `scan_meta.status` is only set to `RUNNING` AFTER `download_incremental_ohlc()` and `get_cached_symbols()` complete (`schedulers/daily_refresh.py`). Since that download is real network I/O across the full universe, it can take minutes. Any frontend polling `/scan-status` during that window — which is exactly what the topbar does every time the page loads — is told the scanner is idle/complete when a scan is, in fact, actively running. This is a backend sequencing issue, not a frontend bug.

The LaunchPad/Alpha Zone pages' own fetches were also instrumented (`frontend_rendered: N` cards logged on every response) to catch "Mongo updated but UI still shows the old count" independently of this — no such gap was found in those fetches themselves; they render exactly what the API just returned.

8. Is polling implemented correctly?

Verdict: mechanically yes. `app/screener/layout.tsx`'s `checkStatus()` polls `/scan-status` every 3 seconds ONLY while status is `RUNNING`, and the interval is cleared on unmount/status change — a clean single-flight lifecycle with no leaked timers or overlapping requests. It faithfully calls `setScanMeta()` with whatever the backend returns, unconditionally. The chain Frontend Poll → Backend Response → React State Update → UI Render has no intermediate logic that could reorder, cache, or delay a value once received.

9. Root cause of every issue found

Issue	Root cause	Evidence
Progress jumps around	No lock around <code>run_daily_scan()</code> ; shared <code>scan_meta</code> doc; <code>\$set</code> overwrites <code>status</code> immediately	Static citation, Section 6
Duplicate scans occurred	<code>force=True</code> bypasses the only guard; two independent trigger buttons	LIVE — observed unprompted, Section 5 & 6
scan-status shows stale data mid-scan	<code>status</code> only flips to <code>RUNNING</code> after download completes	LIVE — 3 checks over 12+ min, Section 7 & 8
Stale in-memory OHLCV on repeated scans	<code>refresh_in_memory_cache()</code> only runs when the dict is empty	Static citation, <code>ohlc_downloader.py:99,119,</code>
Cron can't overlap manual (in THISSession)	Scheduler not installed — logged at startup	Live-captured, startup log
Misleading startup log	<code>main.py</code> always logs "Scheduler: pre-market check..." even when <code>setup_scheduler()</code> returned <code>None</code>	Static citation
Download exception under concurrent runs	There are datetime error inside <code>download_incremental_ohlc()</code> while 2 scans ran in parallel	LIVE — observed, logged in Session 11 CS

10. Recommendations (described only — nothing implemented)

None of the following were built or scaffolded, by design, so the original behavior stayed fully observable for this audit.

- 1 A lock/semaphore (e.g. an `asyncio.Lock` or a Mongo-based mutex document) around `run_daily_scan()` so a second call while one is active is rejected or queued instead of running concurrently.
- 2 Replace the single shared `scan_meta._id="daily_scan"` document with one document per scan (keyed by `scan_id`), so concurrent runs can never overwrite each other's progress field.

- 3 Use an atomic \$inc for symbols_processed instead of re-\$set-ing a locally-tracked counter.
- 4 Set scan_meta.status = RUNNING at the very start of run_daily_scan() (before the download step), not after — closes the stale-status window found in Section 7.
- 5 Consolidate the two independent "Run Scan" buttons/handlers into one shared function so there is only ever one call site to reason about.
- 6 Make get_cached_symbols() aware of whether a fresh download just happened, so a repeat scan in the same process reliably reflects new data.

11. Scanner Timeline (live-captured this session)

Real wall-clock trace from one normal manual click, captured with the instrumentation added in this task (no double-trigger, no manufactured concurrency). This environment's OHLC download is genuine network I/O across the full universe and can run for an extended period; the timeline below reflects the portion actually observed during this investigation, not a simulated/complete run.

Time (UTC)	Event
13:03:53	POST /trigger-scan received currently_active_scans=none no_lock_guard=True
13:03:54	SCAN STARTED scan_id=SCAN-...-6e681b trigger=manual force=True active_before=none
13:03:54	Downloading OHLC... (download_incremental_ohlc()) begins real network sync)
13:03:54 →	Live per-symbol candle sync in progress (confirmed via real data-quality log lines, progressing alphabetically A → K over ~11 m
13:04:xx	GET /scan-status polled — returned the PREVIOUS run's COMPLETED state (symbols_processed=4207, stale) — see Section
13:06:xx	GET /scan-status polled again — identical stale COMPLETED data, still mid-download
13:14:35	A SECOND scan starts UNPROMPTED — SCAN STARTED scan_id=SCAN-...-4ebddc active_before=['SCAN-...-6e681b']
13:14:35	CONCURRENCY WARNING logged — 1 scan(s) already running when this one started
13:14:35 →	Both scans now running their own download_incremental_ohlc() concurrently against the same universe
13:16:04	GET /scan-status re-checked — STILL the same original stale COMPLETED payload, now with TWO real scans active underm
~13:2x	Incremental ingestion failed: "Tz-aware datetime.datetime cannot be converted to datetime64 unless utc=True" — caught by th
~13:2x	SCAN-...-6e681b] In-memory cache has 2068 symbols available → scan_meta set to RUNNING total_symbols=2068
~13:2x	SCAN-...-6e681b] PROGRESS 100/2068 active_scans_now=[BOTH scan_ids] — the race, caught live

This trace independently proves two findings end to end: the Section 7 staleness window (three checks over 12+ minutes, all stale), and the Section 5 concurrency risk (a real second scan starting naturally, unprompted, 11 minutes into the first). Both were captured live in a single continuous observation session.

Additional observation (hedged — plausible, not proven causal): while both scans were mid-download, one hit a real pandas exception — `Tz-aware datetime.datetime cannot be converted to datetime64 unless utc=True` — inside `download_incremental_ohlc()`. The existing (pre-instrumentation) except clause caught it and the scan fell back to whatever was already on disk. Both scans read and write the same `Stock_Data.csv` concurrently, so a partial/interleaved write from one scan landing mid-read of the other is a plausible mechanism — but this was not isolated further (no fixes, no deeper root-causing, per scope) and could equally be a pre-existing data-quality issue unrelated to concurrency. Flagged for follow-up, not asserted as proven.

12. "Run Scan" entry-point audit

Project-wide search for `triggerScan()`, `handleTriggerScan()`, `run_daily_scan()`, and `POST /trigger-scan` turned up exactly two frontend call sites — confirmed by `grep`, not sampled.

Location	Calls Backend?	Duplicate?	Used?
Overview page (app/screener/page.tsx:75)	Yes — <code>screenerService.triggerScan()</code>	Yes (2nd of 2)	Yes — "Run Full Scan" button
Layout topbar (app/screener/layout.tsx:129)	Yes — <code>raw fetch(...trigger-scan)</code>	Yes (1st of 2)	Yes — "Run Scan" button, visible on ever

LaunchPad page	No	—	No — relies on lazy-build / shared cache
Alpha Zone page	No	—	No — relies on lazy-build / shared cache
Technical Scanner page	No	—	No
Volume Scanner page	No	—	No

The two real call sites are independent React components with independent "triggering" state — nothing coordinates them. A user who clicks both (or double-clicks one while the first request is still in flight) queues two full pipeline runs with no server-side deduplication, per Section 5.

Cleanup: every line added for this investigation is tagged `TEMP DEBUG` in both the backend and frontend and can be removed with a single `grep` pass whenever you're ready — none of it changes any return value, computation, or control-flow decision.